scientific reports

Check for updates

OPEN

# Decentralized adaptive task allocation for dynamic multi-agent systems

Elizaveta Tarasova[1]✉, Victoria Erofeeva[1], Oleg Granichin[1] & Kirill Chernikov[2]

**We present a decentralized two-layer architecture for dynamic task assignment in multi-agent systems, designed to operate under partial observability, noisy feedback, and limited communication. The system consists of adaptive controllers that predict task parameters via recursive regression with forgetting and selectively broadcast tasks to a small subset of agents based on relevance and availability. To ensure consistency of task models across the network, we introduce a distributed optimization procedure that combines Simultaneous Perturbation Stochastic Approximation (SPSA) with consensus-based synchronization. The proposed approach enables scalable, online task allocation without centralized coordination. As a representative application, we evaluate the system on simulated workloads involving prompt-based tasks assigned to a diverse set of large language models (LLMs), demonstrating its robustness across varying noise levels, task dynamics, and input arrival patterns.**

Decentralized task allocation in large-scale, heterogeneous multi-agent systems has emerged as a critical challenge in domains ranging from robotic swarms and sensor networks to distributed computing and human–AI collaboration. In such systems, agents independently execute tasks without centralized oversight, enabling resilience to failures, scalability to large populations of agents, and adaptability to dynamic environments. However, achieving efficient and adaptive task distribution under purely decentralized control is nontrivial: agents possess diverse capabilities, tasks exhibit varying urgencies and processing requirements, and timely feedback is often delayed or noisy. Conventional approaches–such as auction-based mechanisms, heuristic protocols, or market-inspired bidding schemes–typically assume static agent performance models or rely on system-wide broadcasts, leading to reduced efficiency and robustness when agent behavior drifts or task types evolve.

Dynamic environments pose three principal challenges for decentralized task allocation:

- Behavioral drift: Over time, an agent's processing speed, reliability, or resource availability may change due to wear, load fluctuations, or context shifts. Static performance models fail to capture such non-stationarity, resulting in mismatches between tasks and agents.
- Non-stationary feedback: Task completion outcomes may not reflect historical patterns–e.g., the same agent may exhibit different success rates under varying network conditions or environmental contexts–requiring continual model updates.
- Delayed observability: In realistic settings, agents' contributions and task outcomes are observed with latency, preventing immediate system-wide adaptation and potentially causing local controllers to operate on stale information.

Addressing these challenges without resorting to centralized coordination requires lightweight, fully decentralized mechanisms for both adaptation and synchronization. In the next section, we review prior works on decentralized allocation schemes and adaptive coordination strategies.

## Related works

Decentralized task allocation in multi-agent systems has been addressed through a wide range of methods. Classical approaches rely on auction-based or rule-driven protocols[1–3], which allow agents to make local decisions with limited coordination. More recent developments include hierarchical or streaming strategies[4,5], enabling large-scale deployment and partial visibility. Industrial studies have also emphasized ontology-based frameworks for adaptive resource management, where reusable task models support flexibility across domains[6].

[1]Saint Petersburg State University, Saint Petersburg, Russia. [2]ITMO University, Saint Petersburg, Russia. ✉email: elizaveta.tarasova@spbu.ru; el.u.tarasova@gmail.com

While effective in static or well-specified environments, such methods often lack robustness under dynamic conditions, noisy feedback, or behavioral drift.

Recent advances in consensus control emphasize prescribed-time convergence and event-triggered/self-triggered mechanisms, ensuring guaranteed agreement within finite horizons while reducing communication load. Dynamic and self-triggered protocols have been proposed for nonlinear MAS, achieving prescribed-time consensus without Zeno behavior[7]. Further extensions address leader-following consensus with input saturation under a time-varying dynamic event-triggered mechanism, providing explicit lower bounds on inter-event intervals[8]. Related studies have also proposed accumulative-error-based event-triggered schemes for discrete-time linear systems, where looped functional methods enable co-design of control gains and event parameters under input saturation[9]. These results underline the importance of resource-efficient and resilient coordination, complementary to decentralized task-allocation research.

Taken together, these studies illustrate the breadth of existing approaches to decentralized task allocation, ranging from control-theoretic consensus protocols to market-based negotiation and machine-learning-driven scheduling. To structure the discussion, we next group related work into several methodological categories–fog computing and distributed scheduling, market-inspired coordination, deep reinforcement learning and embedding-based methods, hybrid heuristic and fuzzy logic schemes, metaheuristic optimization, privacy-aware models, and LLM-based multi-agent coordination. This organization highlights both the diversity of solutions and the common limitations, particularly reliance on static assumptions, centralized training, or the absence of continuous adaptation to drift and delayed feedback.

*Fog computing and distributed scheduling.* The fog-computing paradigm epitomizes the very challenges we target – resource dynamics, unstable performance parameters, and delayed observability – since micro-data centers, routers, and edge devices must self-organize task execution without global coordination[10,11]. We review insights from fog scheduling–including selective task offloading to reduce broadcast overhead, lightweight local performance models to accommodate behavioral) drift, and intermittent synchronization under communication latency–that are critical for decentralized adaptive task allocation research.

Fully decentralized, multi-agent frameworks have been introduced to handle dynamic workloads and unexpected events in fog environments, avoiding single points of failure by local negotiation and self?coordination[12]. In the context of the Internet of Underwater Things (IoUT), a hybrid framework has been developed that combines clustering, ant-colony optimization, and a pseudo-trajectory homotopy method for unmanned marine vehicles (UMVs)[13]. The two-layer architecture with surface and underwater vehicles illustrates distributed task planning under severe communication constraints, which is conceptually close to fog-like environments with intermittent connectivity. In smart manufacturing, hybrid algorithms that combine rule?based heuristics with metaheuristics improve makespan and energy efficiency in fog?enabled pipelines[14]. A recent taxonomy classifies these methods by objectives–latency, energy, or load balance–highlighting delay and energy minimization as most common[11]. Machine learning for resource prediction, classification, and adaptive control is increasingly adopted in fog nodes[15]. Federated and distributed learning extend these gains to wireless?heterogeneous settings, sharing model updates without central aggregation[16]. Fair task distribution has been modelled as a Distributed Envy Minimization Problem (DEMP), where cooperative algorithms ensure Pareto efficiency while minimizing envy among agents with indivisible resources[17].

Most fog scheduling schemes either rely on static heuristics–failing to track long?term performance drift–or incur high communication costs through system?wide broadcasts. For example, dynamic multi?criteria schedulers balance urgency, data size, and complexity but lack decentralized feedback loops and predictive modelling[18].

*Decentralized market-inspired coordination.* We also examine decentralized market?inspired coordination approaches, which apply economic principles to decentralized task allocation. The seminal Contract Net Protocol laid out announce–bid negotiation between task issuers and executors, and has been adapted for smart homes and multi?robot teams[19,20]. Recent auction schemes focus on scalability and resilience to communication delays. Greedy decentralized algorithms enable real?time bidding with minimal overhead[1], while delay?aware frameworks incorporate bid quality and timing into winner selection[3]. Although auctions reduce conflicts and support heterogeneity, they often require centralized bid aggregation or global broadcasts, creating bottlenecks under high churn or partial observability.

*Deep reinforcement learning (DRL) and embedding-based methods.* A twin-delayed DDPG model has been proposed for offloading delay-sensitive tasks in static fog topologies, where each agent learns task-to-node mappings based on fixed observability scopes[21]. GraphSAGE embeddings have been combined with decentralized DRL, but this approach requires a global training phase and prior knowledge of the network graph[22]. Both methods excel in their narrow settings, but struggle to accommodate changing agent sets and partial visibility in real time. Task allocation has also been formulated as a potential game, where reinforcement learning with graph neural networks enables scalable coordination across large agent populations[23]. While effective in structured environments, this approach depends on centralized training and full task observability, limiting adaptability in dynamic and partially observable systems.

*Hybrid heuristic and fuzzy logic schemes.* Hybrid approaches integrate rule?based heuristics or fuzzy logic with metaheuristics or DRL for domain?specific pipelines. For example, fuzzy controllers have been combined with genetic algorithms or DRL for smart manufacturing, typically under the assumption of static network configurations[14,24]. Extensions to dynamic fog workloads have also been proposed using fuzzy rules, but these approaches lack online learning or feedback-driven adaptation[25]. These systems optimize makespan or energy use but do not generalize well to fully dynamic, non?preemptive environments.

*Metaheuristic and optimization-based algorithms.* Metaheuristic methods such as the artificial hummingbird algorithm focus on minimizing global cost metrics under known workload distributions[26]. DCOP?based schedulers solve constraint networks through precomputed bounds and binary constraints[27], while

envy?minimization models ensure fairness among agents with indivisible resources[17]. These techniques provide strong theoretical guarantees but typically require central coordination, offline computation, or persistent agent–task mappings, limiting their applicability in rapidly changing settings.

*Privacy-preserving and energy-aware models.* Secure task assignment via encrypted search protects sensitive constraints but prioritizes confidentiality over allocation efficiency[28]. EcoTaskSched integrates CNNs with BiLSTM to capture spatial and temporal workload features, validated on DeFog workloads and an Azure-based COSCO testbed[29]. While it demonstrates significant energy savings and SLA improvements, it relies on centralized training and fixed task assignment, limiting adaptability to dynamic feedback in decentralized settings. Such approaches are unsuitable when global visibility is unavailable or the environment evolves faster than model retraining cycles.

*LLM-based multi-agent coordination.* Recent studies have explored the use of Large Language Models (LLMs) as autonomous agents capable of self-organizing task assignments through role prompting and self-selection[30]. Such frameworks demonstrate the potential of LLMs for distributed negotiation and resource allocation in multi-agent settings. However, the coordination mechanism remains primarily linguistic and high-level, lacking lower-level adaptive control or predictive task modeling under dynamic feedback and partial observability.

Table 1 summarizes representative decentralized task?allocation systems, highlighting their architectural layers, coordination mechanisms, and key assumptions. Below, we group them by methodology and contrast their limitations with the design goals of our framework.

These prior studies demonstrate substantial progress but still lack mechanisms for fully decentralized, online adaptation under behavioral drift, non-stationary feedback, and delayed observability – challenges that our approach explicitly addresses.

## Research highlight & contributions

This work addresses the problem of decentralized task allocation in large-scale, heterogeneous agent systems. We propose a novel architecture and supporting algorithms that enable scalable, adaptive, and communication-efficient task allocation without centralized coordination.

Across existing paradigms, three recurring limitations remain unresolved:

- Static assumptions. Many approaches require fixed network topologies, predefined offloading points, or full observability.
- Offline training or heavy computation. DRL, embedding, and metaheuristic methods often involve centralized preprocessing or extensive retraining.
- Lack of continuous adaptation. Fuzzy, heuristic, and optimization algorithms rarely incorporate feedback loops to handle behavioral drift and delayed observability.

| System (Reference) | Type/layer | Coordination strategy | Main characteristics | Key differences from our work |
|---|---|---|---|---|
| Ali et al., 2025 [21] | Fog + Multi-agent DRL | TD3-based coordination with delay sensitivity | Centralized training with distributed actors; static fog topology assumed | No adaptation to behaviour drift; requires complete observability and fixed infrastructure |
| Ratnabala et al., 2025 [22] | Decentralized DRL + GraphSAGE | Local DRL agents guided by graph embeddings | Graph-based message passing; learned topology-based scheduling | Requires prior training and stable topology; lacks implicit feedback-driven routing |
| Choppara et al., 2024 [24] | Hybrid Fuzzy + DRL | Rule-based fuzzy coordination with learning refinement | Static rules enhanced by reinforcement tuning; application-specific | Not general-purpose; no adaptation to system dynamics or partial observability |
| Ghafari et al., 2023 [26] | Fog heuristic (metaheuristic) | Centralized hummingbird optimizer | Offline heuristic for energy/delay optimization in fog | No online routing or agent coordination; not applicable in dynamic settings |
| Wang and Li, 2019 [14] | Hierarchical fog-cloud | Top-down scheduling layers | Designed for production pipelines with cloud-fog tiers | Lacks flat agent-controller architecture; not adaptive at task level |
| Netzer et al., 2016 [17] | Decentralized (DCOP) | Iterative negotiation with fairness goals | Focus on envy minimization and Pareto efficiency | No task prediction or learning; assumes stable resource constraints |
| Khan et al., 2025 [29] | Fog–Cloud hybrid ML | CNN+BiLSTM with COSCO+Azure testbed; trained on DeFog workloads | Demonstrates significant energy savings and SLA improvements in fog–cloud IoT systems | Relies on centralized training and fixed task placement; lacks decentralized online adaptation and feedback-driven correction |
| Jin et al., 2025 [25] | Fuzzy-rule based | Adaptive fuzzy coordination under uncertainty | Enhanced fuzzy logic with dynamic rule weights for fog scheduling | No learning component; lacks decentralized feedback adaptation |
| Amayuelas et al., 2025 [30] | Self-organizing LLM system | Autonomic resource allocation via LLM agents | Multi-agent prompting for self-selection and role negotiation | Focus on high-level role assignment; lacks decentralized adaptation or predictive modelling |
| Yang et al., 2025 [13] | IoUT with UMVs (USV+AUV) | Hybrid clustering + ACO + homotopy trajectories | Distributed path planning under underwater communication constraints; multi-layer UMV cooperation | Different domain (marine IoUT) but shares challenges of limited communication and decentralized allocation |
| Xing et al., 2025 [23] | Multi-agent task allocation (Potential games) | Reinforcement learning with GNN-based Q-learning | Task allocation framed as a potential game; scalable to large agent sets | Requires centralized training; assumes stable game formulation and complete task observability |
| This work | Two-layer decentralized fog-inspired | Consensus-based feedback coordination | Online routing via adaptive controllers using SPSA-based consensus correction and relevance scoring | Handles partial visibility, non-stationary task features, and evolving agent feedback |

**Table 1**. Comparison of related systems and our approach.

Our framework directly addresses these limitations through a two-layer decentralized architecture with adaptive controllers and SPSA-based consensus synchronization, enabling fully decentralized, online adaptation in dynamic environments with noisy and delayed feedback.

Specifically, to manage incoming tasks in a decentralized manner, we propose a two-layer architecture. The *outer layer* consists of *adaptive controllers* – dedicated decision-making units responsible for analysing tasks, predicting behavioral) parameters (e.g., expected processing time, urgency), and selecting a relevant subset of agents. The controllers do not directly assign tasks; instead, each task is made visible only to a restricted group of agents, who autonomously decide on execution. This limited visibility enables scalability, supports agent heterogeneity, and avoids full system broadcast.

Conventional task allocation mechanisms – such as auction-based schemes and heuristic protocols – tend to degrade in performance when operating in dynamic environments characterized by changing agent capabilities, evolving task types, or inconsistent feedback. These methods typically rely on fixed rules or assumptions about system behavior, making them vulnerable to behavioral drift, where agent performance varies over time; non-stationary feedback, where task outcomes no longer reflect earlier patterns; and delayed observability, where task results are not immediately available for adaptation. In contrast, our framework incorporates mechanisms for real-time adaptation through localized feedback loops, continuous tracking of behavioral parameters, and coordination among controllers. Each controller incrementally updates its predictive models based on the outcomes of tasks it observes, enabling robust performance under uncertainty and change.

To balance the distribution of task-handling responsibilities among controllers, we apply a load balancing strategy based on a consensus protocol. This mechanism provides a lightweight mechanism that assigns tasks to controllers based on real-time estimates of workload and urgency. It operates without centralized coordination and is resilient to changing network conditions, making it well-suited for dynamic, large-scale systems.

Once a task is assigned to a controller, local models are used to predict relevant parameters (such as expected processing time), and agents are scored based on availability and task fit. Behavioral prediction is performed using recursive models with exponential forgetting, which allow each controller to adapt to drift while using only its local history.

To maintain coherence between local models across controllers, a decentralized synchronization mechanism is introduced. Specifically, we employ decentralized SPSA-based consensus[31], which allows each controller to refine its prediction parameters via randomized local perturbations and shared function-value observations. This synchronization step ensures that controllers do not diverge in their interpretation of similar task types, despite working under decentralized and noisy conditions. Importantly, the applied SPSA-based consensus method demonstrates consistency under almost arbitrary bounded disturbances, in the sense formalized in[32]. This robustness property is especially critical in real-time online systems, where tasks arrive continuously and it is often impossible to repeat experiments or accurately characterize the distribution of uncertainties. The importance of developing methods that remain stable and consistent in such non-reproducible, dynamic environments has been further emphasized in our prior work[33].

Additionally, we evaluate the system in a large-scale simulated setting involving task generation from an LLM, highlighting potential applications in knowledge work (e.g., document generation, translation, and analytical content creation), content routing, and decentralized language service orchestration.

Building on these design choices, the novelty and main contributions of this work are summarized as follows.

- *Online adaptation without centralized retraining.* Unlike conventional methods that assume static models or require periodic offline training, our system explicitly handles behavioral drift, non-stationary feedback, and delayed observability by updating task–agent predictions via localized feedback loops.
- *Two-layer architecture with restricted visibility.* Controllers analyse tasks and expose them only to a limited subset of agents, which improves scalability, reduces overhead, and avoids system-wide broadcasts.
- *Decentralized synchronization via SPSA-based consensus.* We introduce a lightweight mechanism that aligns local predictive models under noise and decentralization, ensuring robustness to uncertainty, an aspect often underexplored in existing approaches.
- *Application to LLM-based workloads.* Beyond synthetic scenarios, we validate the approach in multi-LLM task execution settings, demonstrating applicability to emerging decentralized knowledge-work environments.

## Two-layer decentralized architecture

This section outlines the system structure and task representation, followed by its operational dynamics. The proposed two-layer architecture decouples task interpretation from execution, enabling adaptive coordination among heterogeneous agents with limited visibility and communication.

### System architecture and task representation

We consider a *two-layer decentralized architecture* designed for real-time task processing in an *online setting*. Tasks arrive over time as external requests submitted by clients, services, or sensors. Formally, we denote the task stream as $T = \{T_1, T_2, \dots\}$, an unbounded sequence of tasks arriving at unknown times. Each task $T_i$ is characterized by its type, arrival time $t_i^{arr}$, and other metadata (e.g., length, deadline, constraints). At each time instant, the system has access only to the prefix $\{T_1, \dots, T_i\}$ of tasks that have already arrived by the current time; future arrivals are unknown. This online setting imposes specific constraints on prediction and assignment mechanisms. The overall structure of the system is illustrated in Fig. 1.

The system comprises two logical layers:

- The outer layer consists of adaptive controllers $\mathcal{C} = \{C_1, C_2, \dots, C_m\}$, where each controller $C_j$, $j \in \{1, \dots, m\}$, analyzes tasks, predicts behavioral parameters, selects a set of potential execution agents,

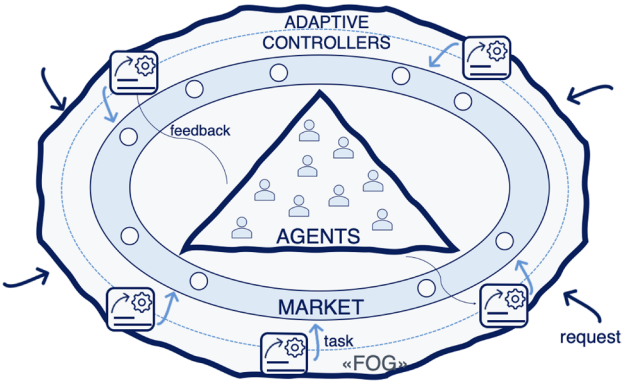



**Fig. 1**. Two-layer decentralized system with adaptive controllers and agent clusters operating under limited visibility and feedback.

and receives feedback. These controllers serve as functional analogues of *fog nodes* in classical fog computing paradigms;
- The inner layer consists of execution agents $\mathcal{E} = \{E_1, E_2, \ldots, E_n\}$, where each agent $E_k$, $k \in \{1, \ldots, n\}$, processes tasks based on limited local information and constrained visibility.

Our system adopts a decentralized flat structure that aligns with fog computing principles without enforcing explicit architectural tiers. We draw a conceptual analogy between *fog nodes* and our cognitive controllers—distributed units operating with limited local visibility and proximity to data sources[10,12]. These controllers perform local prediction, select potential agents, and adapt to feedback through consensus-based updates. This decentralized design improves system responsiveness and robustness under dynamic and heterogeneous conditions[15,16].

Each controller $C_i$ continuously receives feedback from the agents it coordinates. This feedback includes actual task durations and agent responses, allowing the controller to refine its internal models. The adaptation process is online and incremental: prediction errors are tracked over time, and recent data is prioritized via weighted updates. This feedback loop ensures that controllers remain responsive to system dynamics, including shifts in agent behaviour, workload patterns, or resource availability.

In addition to prediction and feedback adaptation, market-based paradigms inspire our system. Cognitive controllers act as task announcers similar to managers in Contract Net Protocol[19], identifying candidate agents and distributing tasks based on local information. While explicit bidding is not used, the interaction structure reflects the principles of distributed negotiation, where agent visibility, task relevance, and predicted performance guide assignment decisions. This hybrid structure allows the system to leverage the robustness of decentralized auction-based methods while maintaining lightweight communication overhead[1,20].

*Task and client representation*
Each task $T_i$ arrives at a time instant $t_i^{arr}$ and is characterized by:

- a task type $type_i \in \mathcal{T}$, where $\mathcal{T}$ is a finite catalogue of distinct task categories (e.g., data-aggregation, image-processing, query-routing, etc.), and each task is assigned a discrete label indicating its semantic class;
- an estimated complexity positive $h(T_i) \in \mathbb{R}_+$;
- a feature vector $\phi_i \in \mathbb{R}^d$ describing the *task profile*, including urgency, cost, agent restrictions, technical requirements, and other abstract attributes;
- an optional *client profile* $\kappa_i$, when the task is issued by a known customer such as a person, organization, or service.

The estimated task complexity $h(T_i)$ is computed by the assigned controller based on task features and system-specific heuristics. It reflects how resource-intensive or challenging a task is expected to be, and depends on the specific application domain. In the simplest case, each task type $type_i$ is associated with a predefined constant complexity value. In more advanced settings, $h(T_i)$ may additionally depend on the task's feature vector $\phi_i$ and, if available, the client profile $\kappa_i$. For example, in systems where tasks originate from known users or services, $h(T_i)$ may incorporate client-specific constraints, behavioral patterns, or historical feedback. This complexity score is used for normalizing prediction errors, adapting scheduling thresholds, and prioritizing execution when necessary. Historical metadata used for prediction may be aggregated either over task types or over specific clients, if available.

Each task is associated with an internally generated deadline $D_i$, interpreted as the latest time by which the task is expected to be completed. The deadline is not externally defined, but rather inferred from:

- the predicted processing time;
- expected agent readiness;

- contextual factors such as task urgency, current system load, and internal service-level policies (SLA: Service-Level Agreement guidelines) that reflect expected timeliness for different types of requests.

Hence, $D_i$ acts as a soft target shaped by the system's internal assessment and temporal constraints.

*Agents and local state*
Execution agents do not have global access to incoming tasks. Instead, each controller selects a subset of agents to whom a task is made visible, based on relevance, availability, and internal policies. This visibility mechanism is analogous to geolocation-based task assignment in ride-hailing systems.

Each agent $E_k$ makes an *autonomous decision* about whether to accept a visible task, based solely on local information and predefined heuristics. In decentralized multi-agent systems, such local decision-making is typically based on utility maximization under partial observability. Common strategies include:

- Greedy selection, based on minimizing estimated cost or finish time[27].
- Earliest Finish Time (EFT), used in dynamic scheduling with local estimates[28].
- Threshold-based filtering, where agents accept tasks only if predicted utility exceeds a threshold [34].
- Auction-style or contract-net mechanisms, enabling agents to bid or negotiate over tasks[2,3].
- Reputation-aware policies, where agents use historical success or delay statistics to guide selection [35].

These decentralized policies enable agents to coordinate without global synchronization, often achieving near-optimal task distribution through emergent behaviour. In our model, agents use a simplified local policy combining the earliest availability estimates and relevance scores provided by the controller.

Tasks are executed in a *non-preemptive* manner: once an agent starts executing a task, it must complete it without interruption or reassignment.

The state of an agent includes:

- availability status $s_k(\text{time}) \in 0, 1$;
- queue of accepted tasks $Q_k$ (ordered by assignment);
- start time of the currently executed task $t_{i'}^{\text{start}}$;
- predicted processing time $\hat{p}_{i'}$ for the current task $T_{i'}$, provided by the controller;
- prior expected finish time $\hat{v}_{i'} = t_{i'}^{\text{start}} + \hat{p}_{i'}$;
- trust coefficient $\rho_j \in [0, 1]$ assigned to controller $C_j$ that scheduled the task;
- auxiliary feature vector $\eta_k \in \mathbb{R}^q$, capturing agent-specific characteristics that affect task eligibility and relevance.

Values in $\eta_k$ may include agent-specific processing speed, cognitive load, psychotype (in human-in-the-loop settings), fatigue, hardware limits, and other contextual properties.

In general, we define

$$t_k^{\text{free}} = \begin{cases} time, & Q_k = \emptyset, \\ \displaystyle\sum_{T_i \in Q_k \setminus \{T_{i'}\}} \hat{p}_i \; + \; t_{i'}^{\text{start}} \; + \; \rho_j \cdot \hat{p}_{i'} \; + \; (1 - \rho_j) \cdot \max\big(0,\; time - \hat{v}_{i'}\big), & \text{otherwise.} \end{cases} \tag{1}$$

Equation 1 estimates the expected release time of agent $E_k$ by combining:

- $\sum_{T_i \in Q_k \setminus \{T_{i'}\}} \hat{p}_i$: forecast durations of pending tasks, assumed reliable until execution begins;
- $t_{i'}^{\text{start}} + \rho_j \cdot \hat{p}_{i'}$: baseline estimate for the current task, weighted by the trust coefficient $\rho_j \in [0, 1]$;
- $(1 - \rho_j) \cdot \max(0,\, time - \hat{v}_{i'})$: corrective term if execution already exceeds the forecasted finish $\hat{v}_{i'}$.

All quantities are locally available to the agent (queue contents, predicted durations, start times, local clock), so the formula is feasible without global synchronization. Correctness follows from two cases: if $time \leq \hat{v}_{i'}$, the correction vanishes and the estimate coincides with the forecast; if $time > \hat{v}_{i'}$, the correction prevents underestimation by ensuring the release time is no earlier than the actual elapsed execution. This hybrid formulation ensures feasibility in edge cases while preserving adaptive correction for tasks under execution.

## Overall system workflow

This subsection describes the sequence of steps involved in processing a task within the proposed decentralized two-layer architecture. The operation of the system is organized as follows:

**Step 1: Task Arrival & Controller Assignment.** A new task $T_i$ arrives at the system at the time $t_i^{arr}$ (e.g., generated by an external client, service, or sensor). It initially becomes available in a local fog-accessible region and enters a queue of one of the nearby controllers. The final assignment to a responsible controller $C_j \in \mathcal{C}$ is determined via a decentralized local voting protocol, which enables dynamic reallocation across controller queues based on current load and predicted readiness. This distributed coordination continues until one controller becomes ready to expose the task to a subset of agents. This mechanism ensures:

- load balancing among controllers,
- robustness to partial communication,
- decentralized operation based on local information only.

**Step 2: Task Parameter Prediction.** The assigned controller $C_j$ analyzes the task using the profile $\phi_i$, along with historical execution data and local models. It predicts key behavioral parameters:

- expected processing time $\hat{p}_i$,
- acceptable wait time $\hat{w}_i$,
- internal deadline $D_i$,
- other task-specific parameters, depending on the system design.

The prediction process combines global models and local adaptation, allowing both long-term patterns and short-term dynamics to be captured.

**Step 3: Agent Evaluation and Cluster Formation.** The controller evaluates all agents $E_j \in E$ using two key metrics:

- $R_{j,i}$ denotes the relevance score quantifying the compatibility between task *j* and agent *i*, typically computed from the agent's profile $\eta_i$, where *i* indexes agents and *j* indexes tasks;
- availability score $A_{j,i}$, estimating the likelihood of task completion before the deadline $D_i$.

These values are used to perform clustering in the $(R_{j,i}, A_{j,i})$ space, and a subset of candidate agents is selected accordingly.

**Step 4: Task Dissemination to Agent Subset.** The task is made visible only to the selected group of agents. Each agent:

- receives the task through the controller,
- evaluates it autonomously based on local decision rules,
- may use heuristics such as earliest expected completion or minimal current workload.

**Step 5: Execution and Feedback.** Upon task completion, the selected agent reports back to the controller:

- actual processing time,
- actual finish time,
- task status (e.g., successful, interrupted, or delayed).

This feedback is used to update the controller's prediction models and refine agent evaluations. Additionally, controllers periodically synchronize their internal models using an SPSA-based consensus optimization procedure. The detailed mechanisms implemented by the controllers, including task parameter prediction, agent cluster selection, and global model correction via stochastic optimization, are described in the next section.

## Decentralized adaptive controllers

### Task assignment via load balancing protocol

When a new task $T_i$ arrives into the system, it must be assigned to one of the adaptive controllers $C_j \in \mathcal{C}$. This controller becomes responsible for analysing the task, predicting its parameters, and selecting a relevant pool of execution agents. Since the system operates in an online decentralized regime without a central dispatcher, task allocation must rely solely on local observations and limited communication between neighbouring controllers.

To redistribute incoming tasks among controllers and balance the queues of tasks awaiting assignment, we employ a load balancing protocol. This protocol is an iterative and fully distributed mechanism originally designed for multi-agent systems operating under dynamic and stochastic communication topologies. The protocol enables controllers to reach agreement on load allocation without centralized coordination, while remaining robust to partial observability, communication delays, and asynchronous updates.

Applying this strategy requires first defining what constitutes the load of a node (controller), followed by a description of the load balancing mechanism.

*Controller communication.* We assume that controllers are able to exchange control information (e.g., workload estimates, task-urgency metrics) and collaboratively rebalance task assignments among neighboring controllers. In fog-computing environments, permanent all-to-all connections are typically infeasible: links may be unavailable at certain times due to bandwidth limitations, interference, or traffic constraints. Even when multiple neighbors are accessible, maintaining simultaneous communication with all of them is often impractical. Instead, it is convenient to model inter-controller communication through a time-varying undirected graph $\mathcal{G}(t) = (\mathcal{C}, \mathcal{E}(t))$, where active links represent the subset of neighbors exchanging information at time *t*. Randomized neighbor selection is commonly used in this context to limit communication overhead, which is consistent with the justification for randomized topologies discussed in[31].

The communication weights are captured by an adjacency matrix $B(t) = [b^{j,j'}(t)] \in \mathbb{R}^{m\times m}$, defined as:

$$b^{j,j\prime}(t) = \begin{cases} \dfrac{2}{\deg(j) + \deg(j\prime)}, & \text{if } (j, j\prime) \in \mathcal{E}(t), \\ 0, & \text{otherwise.} \end{cases}$$

The expected adjacency matrix $B_{\text{av}} = \mathbb{E}[B(t)]$ represents the averaged communication graph $\mathcal{G}_{\text{av}}$, where the weight of each edge corresponds to its expected presence over time. The graph $\mathcal{G}_{\text{av}}$ is assumed to be connected, enabling long-term coordination through distributed updates.

*Controller state.* At each time instant *t*, controller $C_j$ estimates its current queue size $q_j(t)$ as the weighted sum of all tasks currently assigned to it:

$$q_j(t) = \sum_{T_k \in \mathcal{A}_j(t)} (1 + \lambda \cdot \varsigma_k),$$

where $\mathcal{A}_j(t)$ is the set of tasks assigned to controller $C_j$, $\varsigma_k$ denotes the urgency of the task $T_k$, and $\lambda \geq 0$ is a system parameter controlling the contribution of urgency to the load metric. If task urgency is not defined in the system, this reduces to simple task counting: $q_j(t) = |\mathcal{A}_j(t)|$ (i.e., the number of tasks in the set $\mathcal{A}_j(t)$).

This metric is justified by the fact that all tasks incur a fixed computational cost for prediction on the controller, since both the number of features and the number of predicted parameters are constant across tasks.

Next, we introduce the state of controller *j* at time instant *t* as

$$x_j(t) = \frac{q_j(t)}{\bar{p}_j},$$

where $p_j$ is the average productivity of controller *j*. The state indicates the processing time of the tasks waiting in the queue of controller *j*.

Let $\tilde{x}_j(u^j|t+1)$ be the time required to complete all tasks in the queue of controller *j* if rebalancing strategy $u^j$ is chosen at time instant *t*:

$$\tilde{x}_j(u^j|t+1) = x_j(t) - \bar{p}_j + \frac{z_j(t)}{\bar{p}_j} - \frac{u_j}{\bar{p}_j},$$

where $z_j(t)$ is the amount of new incoming job packages received by controller *i* during time interval $[t; t+1]$.

*Load balancing protocol.*

We seek an optimal task-distribution protocol by equalizing loads across controllers. Define

$$F(\mathrm{u}|t) = \frac{1}{2} \sum_{j,j' \in \mathcal{N}} b^{j,j'} \Big( \tilde{x}_j(u_j|t+1) - \tilde{x}_{j'}(u_{j'}|t+1) \Big)^2,$$

where $b^{j,j'}$ is the $(j, j')$ entry of the adjacency matrix *B*(*t*), and $\mathrm{u} = [u_1, \ldots, u_n]^\top$ collects controller inputs.

A gradient-based feedback law for $F(\mathrm{u}|t)$ would be

$$u^j(t) = \alpha \sum_{j' \in \mathcal{N}_j(t)} b^{j,j'} \Big( \tilde{x}_j(u_j|t+1) - \tilde{x}_{j'}(u_{j'}|t+1) \Big),$$

where $\alpha > 0$ is the consensus step-size, $\mathcal{N}_j(t) = \{j' \in \mathcal{N} : (j', j) \in \mathcal{E}(t)\}$ is the set of *neighbors*.

Since $\tilde{x}_j(\cdot|t+1)$ cannot be computed directly, we replace it with local measurements $y^{j,j}(t)$ and neighbor measurements $y^{j,j'}(t)$. The resulting rebalancing protocol becomes

$$u_j(t) = \alpha \sum_{j' \in \mathcal{N}_j(t)} b^{j,j'} \big( y^{j,j}(t) - y^{j,j'}(t) \big).$$

The rebalancing inputs $u_j(t)$ can be computed via distributed consensus protocols, for example the Local Voting Protocol (LVP)[36] or its accelerated version (ALVP)[37]. These protocols are well suited to our decentralized setting for several reasons. First, both LVP and ALVP require only neighbour-to-neighbour exchanges of load estimates, eliminating the need for any central coordinator or global broadcasts. Second, under mild connectivity assumptions on the communication graph, LVP converges to a balanced load distribution, while ALVP achieves faster convergence rates and improved transient performance. Third, by using constant or adaptively tuned step-sizes, these protocols maintain stability and responsiveness in the presence of time-varying workloads and link delays. Finally, each controller's update involves only simple weighted sums and local measurements, making LVP and ALVP computationally lightweight and easy to implement in resource-constrained environments. Together, these properties make LVP and ALVP ideal choices for computing $u_j(t)$ in our fully decentralized, dynamic multi-agent framework.

## Task parameter adaptation via recursive regression with forgetting

After tasks have been assigned and rebalanced among controllers using the protocol above, each controller proceeds to predict the key parameters of its tasks – such as expected processing time $\hat{p}_i$ and wait tolerance $w_i$ – using the adaptive techniques detailed in this section.

To adaptively estimate task parameters, each controller $C_j$ maintains a local linear model updated using a recursive forgetting-based regression scheme. The model assumes a prediction of the form:

$$\hat{p}_i = \phi_i^\top \theta_j,$$

where $\phi_i \in \mathbb{R}^d$ is the feature vector for task $T_i$ and $\theta_j \in \mathbb{R}^d$ is the weight vector maintained by the controller $C_j$.

After observing the actual processing time $p_i^{\text{real}}$, the controller updates its model by minimizing a quadratic residual functional with exponential forgetting:

$$J_j^{(t)}(\theta) = \sum_{s=1}^{t} \beta^{t-s} \cdot \left(p_s^{\text{real}} - \phi_s^\top \theta\right)^2,$$

where $\beta \in (0, 1]$ is the forgetting factor, which assigns more weight to recent observations.

The model parameters are updated recursively as follows:

$$\theta_j^{(t+1)} = \theta_j^{(t)} - \Gamma_j^{(t)} \cdot \phi_t \cdot \left(p_t^{\text{real}} - \phi_t^\top \theta_j^{(t)}\right),$$

where $\Gamma_j^{(t)} \in \mathbb{R}^{d \times d}$ is an auxiliary matrix updated by:

$$\Gamma_j^{(t+1)} = \frac{1}{\beta}\left(\Gamma_j^{(t)} - \frac{\Gamma_j^{(t)} \phi_t \phi_t^\top \Gamma_j^{(t)}}{\beta + \phi_t^\top \Gamma_j^{(t)} \phi_t}\right).$$

The initialization is set to $\theta_j^{(0)} = 0$ and $\Gamma_j^{(0)} = \delta \cdot I_d$ for a small $\delta > 0$.

This update corresponds to the standard Recursive Least Squares (RLS) algorithm with exponential forgetting. Each controller independently applies this scheme to estimate local parameters $\theta_j$ based on its own stream of task observations.

The resulting local model captures short-term task dynamics and adapts to parameter drift. These local estimates are subsequently synchronized across controllers using the decentralized *SPSA+consensus* algorithm described in the next section, ensuring network-wide coherence without requiring gradient access or centralized coordination.

This scheme allows for robust local tracking of parameter drift using a lightweight update per task. The adapted vector $\theta_j$, representing the local model parameters of the controller $C_j$, undergoes periodic synchronization with other controllers using a decentralized variant of the SPSA-based consensus algorithm. This process is analogous to distributed learning, where each node maintains its own model and parameters are synchronized across the network without a central coordinator.

A similar online adaptation procedure is applied to estimate the wait tolerance $w_i$, using $w_i^{\text{real}}$ in place of $p_i^{\text{real}}$.

## Global parameter correction via decentralized SPSA

Having described how each controller predicts task parameters locally, we now turn to ensuring that these prediction models remain coherent across the network. To enable consistency across task parameter models maintained by distributed controllers, we employ the Simultaneous Perturbation Stochastic Approximation-based consensus method. This approach allows for gradient-free optimization of prediction models using only two observations of the objective per iteration, making it highly suitable for decentralized and bandwidth-constrained systems.

We introduce two quality functionals to assess the accuracy of our task-parameter predictions. Recall that $\phi_i$ denote the feature vector of task $T_i$. We introduce a $\sigma$-algebra $\{\mathcal{F}_\tau\}_{\tau \geq 1}$, where $\mathcal{F}_{\tau-1}$ contains all tasks seen, measurements made, and estimates computed up to iteration $\tau - 1$. Then each controller's cost functional can be written as a conditional expectation on its information set. By optimizing these conditional expectations, each controller seeks the best prediction parameters in light of all available information, naturally accounting for both variability in tasks and measurement noise.

The first functional evaluates processing-time prediction error, with greater penalty for short or simple tasks:

$$F^{\text{proc}}\left(\theta^{(p)}\right) = \sum_{j \in C} \mathbb{E}_{\mathcal{F}_{\tau-1}}\left[\left(\frac{\phi_i^\top \theta^{(p)} - p_i^{\text{real}}}{h(T_i)\, p_i^{\text{real}} + \epsilon}\right)^2\right], \tag{2}$$

where $p_i^{\text{real}}$ is the observed processing time, $h(T_i)$ measures task complexity, and $\mathbb{E}_{\mathcal{F}_{\tau-1}}$ denotes the conditional expectation.

The second functional measures prediction accuracy for task wait tolerance, with an insensitivity zone of width $\delta > 0$:

$$F^{\text{wait}}\left(\theta^{(w)}\right) = \sum_{j \in C} \mathbb{E}_{\mathcal{F}_{\tau-1}}\left[\left(\max\{\,|\phi_i^\top \theta^{(w)} - w_i^{\text{real}}| - \delta,\, 0\}\right)^2\right], \tag{3}$$

where $w_i^{\text{real}}$ is the actual wait tolerance.

Both functionals involve expectations over locally observed, noisy task outcomes. Moreover, controllers cannot compute analytical gradients due to partial observability, communication constraints, and dynamic workloads. Therefore, we employ a zero-order (gradient-free) method – SPSA-based consensus – to optimize these objectives in a decentralized way using only inexpensive function evaluations.

We now present a decentralized algorithm using only noisy zeroth-order observations and neighbor-to-neighbor communication. The method solves two parallel optimization problems of the form $\arg\min_{\theta^{(p)}} F^{\text{proc}}(\theta^{(p)})$ and $\arg\min_{\theta^{(w)}} F^{\text{wait}}(\theta^{(w)})$, corresponding to processing time and wait tolerance

prediction. Each task leads to two separate parameter vectors, updated independently but using the same decentralized protocol. The method combines Simultaneous Perturbation Stochastic Approximation (SPSA) with a consensus protocol to achieve scalable, communication-efficient optimization.

Let each controller $j \in \mathcal{N}$ maintain an estimate $\widehat{\theta}_j(\tau) \in \mathbb{R}^d$. At iteration $\tau = 1, 2, \ldots$, controller $j$ draws an i.i.d. perturbation $\Delta_j(\tau)$ from a symmetric distribution and uses a perturbation scale $\beta_\tau > 0$. Given a constant step-size $\alpha > 0$ and consensus gain $\gamma > 0$, the update proceeds as follows:

Perturbation and measurement:

$$x_j(2\tau) = \widehat{\theta}_j(2\tau - 2) + \beta_\tau \, \Delta_j(\tau),$$
$$x_j(2\tau - 1) = \widehat{\theta}_j(2\tau - 2) - \beta_\tau \, \Delta_j(\tau),$$
$$y_j(2\tau) = F_j\big(x_j(2\tau)\big) + v(2\tau), \quad y_j(2\tau - 1) = F_j\big(x_j(2\tau - 1)\big) + v(2\tau - 1),$$

Estimate copy:

$$\widehat{\theta}_j(2\tau - 1) = \widehat{\theta}_j(2\tau - 2),$$

Gradient-free update with consensus:

$$\widehat{\theta}_j(2\tau) = \widehat{\theta}_j(2\tau - 1) - \alpha \left( \frac{y_j(2\tau) - y_j(2\tau - 1)}{2\,\beta_\tau} \, \Delta_j(\tau) + \gamma \sum_{j' \in \mathcal{N}^j(\tau)} b^{j,j'} \left( \widehat{\theta}_j(2\tau - 1) - \tilde{\theta}_{j,j'}(2\tau - 1) \right) \right),$$

where:

- $F_j(\cdot)$ is controller $j$'s local cost functional, either $F^{\mathrm{proc}}$ or $F^{\mathrm{wait}}$, depending on the model being updated.
- $\tilde{\theta}_{j,j'}(2\tau - 1)$ is the (possibly noisy) estimate received from neighbor $j'$.
- $v(\tau)$ is the oracle noise.*Algorithm description.*

(a) *SPSA perturbation:* Evaluations $y_j(2\tau)$ and $y_j(2\tau - 1)$ yield a stochastic finite-difference estimate of the directional derivative along $\Delta_j(\tau)$.
(b) *Gradient-free update:* The term $\frac{y_j(2\tau) - y_j(2\tau - 1)}{2\,\beta_\tau} \, \Delta_j(\tau)$ approximates the true gradient into direction $\Delta$.
(c) *Consensus correction:* The sum $\sum_{j'} b^{j,j'} (\widehat{\theta}_j(2\tau - 1) - \tilde{\theta}_{j,j'}(2\tau - 1))$ implements a neighbor-based consensus step, aligning estimates under limited communication.

By alternating zeroth-order gradient approximation with lightweight consensus steps, this algorithm finds a minimizer across the network using only local measurements and peer exchanges.

In contrast to gradient-based methods requiring assumptions on the noise distribution, our SPSA-based consensus approach only assumes that the noise is bounded: deterministically bounded in the case of deterministic disturbances, or mean-squared bounded, regardless of internal dependence structure, in the case of random noise. This enables robust operation in realistic decentralized environments without the need to specify a probabilistic noise model.

## Theoretical convergence guarantee

In the proposed architecture, each controller *j* performs local optimization using a gradient-free SPSA update with an additional consensus correction from neighboring controllers. Although the observations may be noisy, Appendix A shows that the algorithm remains stable as long as the noise is bounded in magnitude, regardless of its distribution. This is particularly important for uncertain environments, such as human-centered or data-driven systems, where the noise model may be unknown.

We build upon the theoretical framework of[31], which provides convergence guarantees for distributed SPSA algorithms with consensus dynamics under mild assumptions. The following theorem presents an adaptation of the main result from that work applied to our setting.

Define the problem-dependent constants,

$$L^{(p)} := \frac{2R_\phi^2}{\epsilon_0^2}, \qquad L^{(w)} := 4R_\phi^2, \qquad g_{2(p)}^2 := \frac{4}{\epsilon_0^4}\left(R_\phi^2 R_\theta + R_\phi P_{\max}\right)^2, \quad g_{2(w)}^2 := 4R_\phi^2\left(R_\phi R_\theta + W_{\max} + \delta\right)^2,$$
$$\bar{\lambda}_2 := \lambda_2(I - B_{\mathrm{av}}), \qquad c_1 = L_\star + 2\gamma, \qquad L_* = \{L^{(w)}, L^{(p)}\}, \qquad c_2 = g_2^2 + d\left(\sigma_v^2/\beta^2 + L_\star^2\beta^2\right), \qquad c_m = \tfrac{2}{\bar{\lambda}_2} \tag{4}$$

where $B_{\mathrm{av}} = \mathbb{E}[B_\tau]$ is the averaged consensus matrix.

*Theorem 1* For the loss functions (2) and (3) with the constants defined in (4), if a constant step-size $\alpha$ satisfies the inequality

$$\alpha < \frac{\bar{\lambda}_2}{c_1 + \sqrt{c_2 c_m}},$$

with constants $c_1, c_2, c_m, \bar{\lambda}_2$ determined by the Lipschitz continuity, gradient bounds, and network topology, then the sequence of estimates of distributed SPSA algorithm with consensus satisfies the following bound:

$$\limsup_{\tau\to\infty} \mathbb{E}\left\|\boldsymbol{\theta}_{2\tau}^{(c)} - \boldsymbol{\theta}^*\right\|^2 \;\le\; \frac{g_2^2\alpha^2}{\bar{\lambda}_2^2}\cdot C,$$

where $\boldsymbol{\theta}_{2\tau}^{(c)}$ is the average iterate across controllers at even iteration $2\tau$, and $C$ is a constant depending on the network size and algorithm parameters.

**Proof** To prove this theorem, it is necessary to verify that Assumptions 1–5[31] are satisfied; the remaining assumptions 6-8 hold by definition. A detailed proof can be found in the Appendix. □

## Experimental validation through LLM-based scenarios

To assess the effectiveness of our approach in LLM-related settings, we simulate a decentralized two-layer system operating on batches of prompts processed by large language models (LLMs). The architecture is not tailored to LLMs per se but supports their heterogeneity and dynamic behavior. The goal of this evaluation is to demonstrate how adaptive controllers can learn task-agent mappings, coordinate assignments, and maintain accurate performance predictions across diverse prompt types and execution conditions.

The simulation environment involves three key components:

- **Tasks**, grouped into batches of prompts with varied structure, semantic categories, and complexity levels. Each task is described by a feature vector used in local prediction models.
- **Agent-executors**, instantiated as heterogeneous LLMs with different capabilities and success rates across prompt types. These agents serve as the lower layer of the system.
- **Adaptive controllers**, forming the upper coordination layer, responsible for predicting task parameters, selecting candidate agents, and coordinating assignment using local information and decentralized consensus.

All modelled configurations employ an adaptation algorithm based on Simultaneous Perturbation Stochastic Approximation (SPSA) with a consensus mechanism presented above. Gradient estimation follows a two-point scheme applied at each parameter update step. The experiments use the following hyperparameters: step-size $\alpha = 0.01$, perturbation scale $\beta = 0.1$, consensus step-size $\gamma = 0.02$, and parameter updates are performed every 10 tasks. These values provide a balance between learning stability and responsiveness to local variations in the environment.

The following sections describe two independent simulation scenarios, differing in the realism of agent and task representations, as well as in how relevance and execution time are defined. The first scenario models a synthetic environment with abstract agents and simulated execution times. The second involves real-time interaction with LLM APIs and measurement of actual task completion times; this scenario also includes a comparison between the proposed method and a round-robin baseline.

### Synthetic simulation

To evaluate the core properties of our task allocation system in a controlled setting, we conducted a synthetic simulation. The environment allowed us to explicitly define task characteristics, agent capabilities, and observation noise, enabling a transparent assessment of system behaviour under varied conditions.

**Task and agent setup.** Each task was represented as a prompt of a certain type, sampled from one of six categories: *Data Analysis*, *Programming*, *Translation*, *Summarization*, Q&A, and *Creative Writing*. Tasks arrived dynamically in batches of size 1 to 8, and each batch could contain a heterogeneous mix of task types. Each task was associated with a release time $r_i$, complexity score $h(T_i)$, and type $\text{type}_i$. The feature vector $\boldsymbol{\phi}_i$ was generated as a real-valued vector of fixed dimensionality.

The system simulated five synthetic agents representing major LLM classes: `GPT-4`, `Claude-3.5`, `Gemini-1.5`, `LLaMA-3`, and `Mistral-7B`. These agents acted as behavioural proxies with adaptive performance characteristics. Rather than using static efficiency profiles, each agent's success probability and response latency were dynamically updated during the simulation based on observed task outcomes.

**Controller logic.** Four adaptive controllers were used, connected in a sparse communication graph. For each task, a controller predicted the expected execution time $\hat{p}_i$ and acceptable waiting window $\hat{w}_i$. The deadline was then computed as $D_i = r_i + \hat{w}_i + \hat{p}_i$.

Based on availability and current relevance, a candidate pool of agents was formed. The controller did not assign a specific agent; instead, agents in the pool independently claimed tasks, mimicking multi-agent selection dynamics. Controllers also coordinated through a Local Voting Protocol (LVP) for distributing tasks among themselves. Model parameters were updated via SPSA with consensus.

The actual execution time $p_i^{\text{real}}$ is then simulated based on the prompt's textual length and declared complexity using the following formula:

$$p_i^{\text{real}} = \max\left(0.0,\ \min\left(1.0,\ 0.3\cdot\frac{\text{len}(T_i)}{1000} + 0.7\cdot\frac{c_i}{10} + \varepsilon\right)\right),$$

where $\text{len}(T_i)$ is the number of characters in the prompt text $T_i$, $c_i$ is the complexity level (ranging from 1 to 10), and $\varepsilon \sim \mathcal{U}(-0.05,\ 0.05)$ is a uniformly distributed noise term. The result is clipped to the interval $[0,\ 1]$ to ensure consistency with the normalized task durations.

**Simulation Goals.** This scenario was designed to assess whether adaptive controllers could learn accurate task-agent mappings, whether the system could maintain consistent success rates across task types, and whether

the prediction of execution time improved through adaptation. We also monitored agent-level specialization and task load distribution as indicators of emergent coordination.

Figure 2a shows the distribution of task assignments across agents. The majority of tasks were handled by `GPT-4` (33.3%), followed by `Claude-3.5` (21.4%), `LLaMA-3` (19.2%), and `Gemini-1.5` (19.0%). `Mistral-7B` handled only 7.1% of all tasks. This reflects the system's preference for agents with higher dynamic relevance and success history. Since controllers form pools rather than making hard assignments, more capable agents are implicitly favoured over time.

Figure 2b presents the success rate by task type. Each task is considered successful if it is completed before its predicted deadline. The success rate for a specific task type $type_i$ is then defined as:

$$\text{SuccessRate}(type) = \frac{1}{|\mathcal{T}_{type}|} \sum_{i \in \mathcal{T}_{type}} \text{success}_i, \quad \text{success}_i = \begin{cases} 1, & \text{if } t_i^{star} + p_i^{real} \leq D_i, \\ 0, & \text{otherwise,} \end{cases}$$

where $\mathcal{T}_{type}$ denotes the set of all tasks of *type*, $t_i^{star}$ is the actual task starting time.

The average success rate across all types was 83.4% (dashed line). The highest values were observed for *Summarization* (92.7%) and *Data Analysis* (91.3%), while more challenging categories such as *Programming* (71.6%) and *Creative Writing* (78.8%) exhibited higher variability. This suggests that the system is robust when handling structurally stable or numerically focused tasks, but more sensitive to agent quality on creative or logic-driven prompts.

We further analyse agent specialization by computing task-type-specific effectiveness. For each agent *a* and task type *t*, we define the effectiveness as:

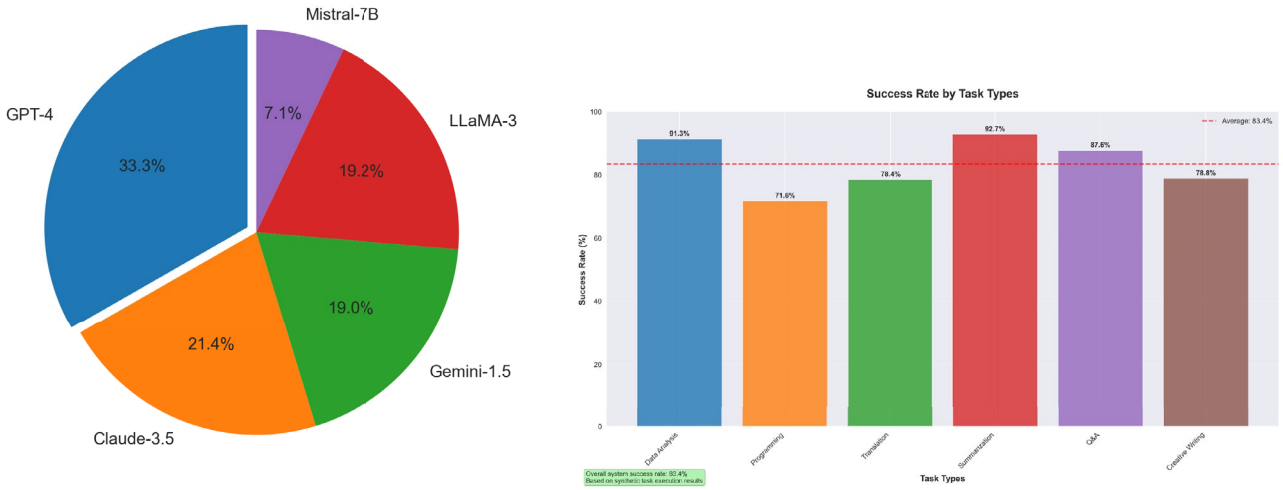
$$\text{Eff}_{k,type} = \frac{1}{|\mathcal{T}_{k,type}|} \sum_{i \in \mathcal{T}_{k,type}} \frac{\text{success}_i}{p_i^{\text{real}}},$$

where $\mathcal{T}_{k,type}$ is the set of all tasks of *type* processed by the agent *k*, and $p_i^{\text{real}}$ is the actual execution time of the task *i*. The result captures both success and speed – lower durations and higher completion likelihoods increase effectiveness. The values are normalized during post-processing to enable heatmap visualization.

Figure 3 illustrates the normalized effectiveness of each agent per task type. Each value captures a combined measure of task success and execution speed. Clear specialization patterns emerge: `GPT-4` performs best on *Programming* (0.90), `Claude-3.5` leads on *Creative Writing* (0.87), and `Gemini-1.5` is strongest on *Data Analysis* (0.71). In contrast, `Mistral-7B` shows consistently low effectiveness, particularly on open-ended tasks. These patterns confirm that the system not only adapts to agent-specific capabilities but also leverages them in decision-making.

Figure 4 evaluates the prediction accuracy of execution time. The left panel compares predicted and actual durations, with points clustered near the identity line, indicating strong alignment. The right panel shows the distribution of relative errors, with a mean error close to zero. The distribution is symmetric and centred, suggesting no systematic over- or underestimation. These results indicate that SPSA-based consensus adaptation allows controllers to maintain reliable and unbiased predictions even under moderate noise.

**Summary.** The synthetic simulation confirms that the proposed system operates robustly under uncertainty. Controllers learn accurate agent-task associations, adapt relevance values based on outcome feedback, and produce reliable execution time estimates. Task assignment naturally shifts toward more competent agents,




**(a)** Distribution of assigned tasks across agents

**(b)** Success rate per task type

**Fig. 2**. System-level behaviour: task allocation and success rates.

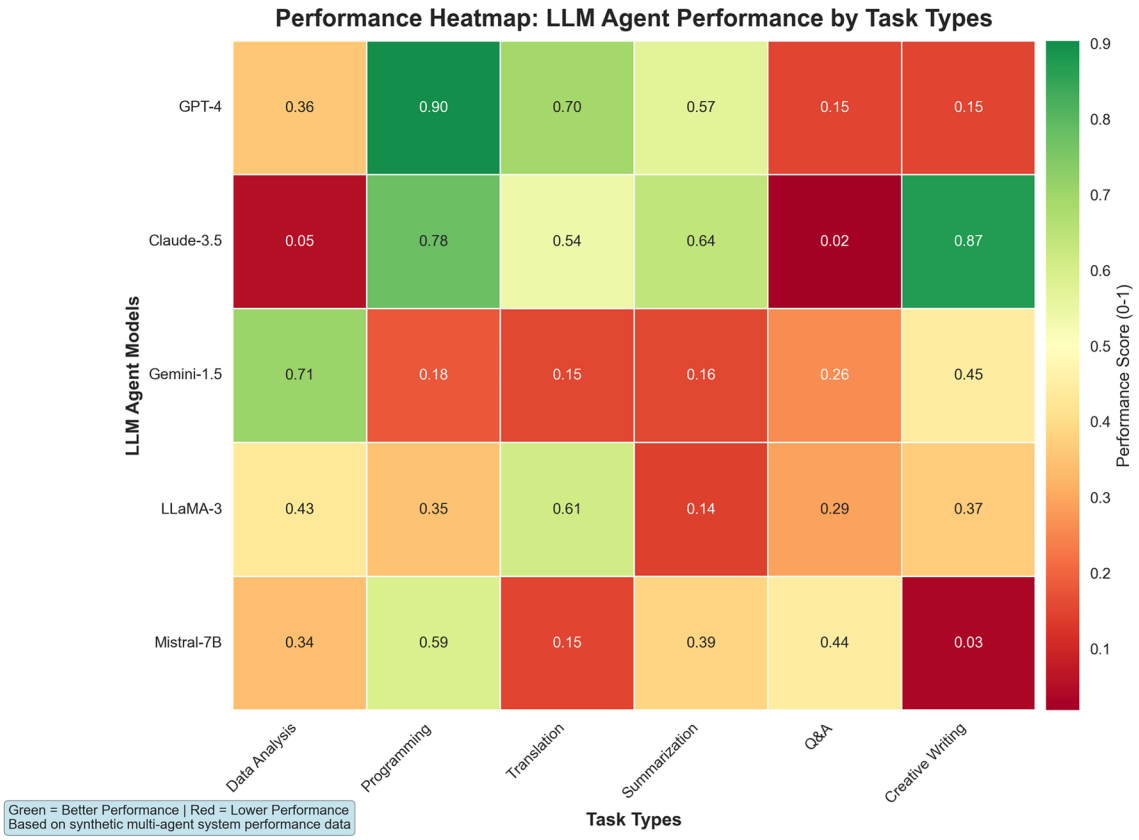



**Fig. 3**. Performance heatmap: normalized effectiveness of each agent across task types.

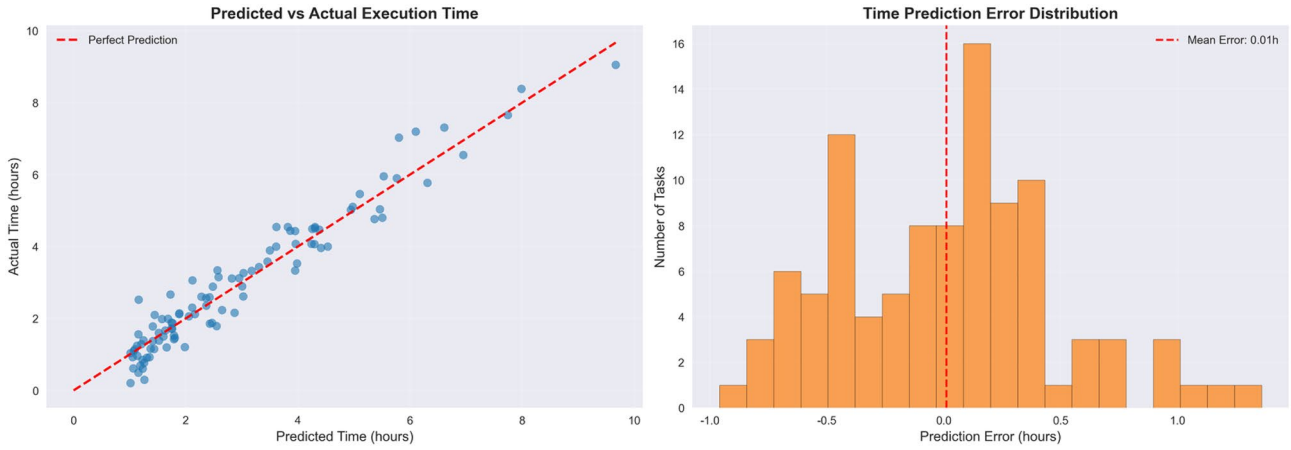



**Fig. 4**. Prediction quality: actual vs. predicted execution times (left); distribution of prediction error (right).

and task categories with predictable structure are handled most reliably. Overall, the experiment validates the learning dynamics and coordination mechanisms embedded in the system.

## Evaluation with realistic LLM heterogeneity

This section demonstrates how the proposed system performs under conditions that mimic real-world heterogeneity in large language model (LLM) capabilities. Unlike the first simulation, where agent performance was synthetically generated, this experiment is based on a fixed effectiveness matrix derived from existing empirical studies. For each task type and model, we predefined a success probability that reflects how likely a given LLM is to successfully complete a task of a specific category. These probabilities are not learned or adapted during simulation but serve as the hidden ground truth used to compute actual task outcomes. The controllers,

unaware of these true capabilities, must learn to estimate and leverage them during runtime based solely on feedback from completed tasks.

The simulation includes six LLM agents: GPT-4, Claude-3.5, Gemini-1.5, LLaMA-3, Mistral-7B, and GPT-3.5. Tasks are sampled from eight categories: Math, Code, Text, Analysis, Creative, Explanation, Research, and Planning. These types reflect a broad range of cognitive demands and linguistic complexity, allowing us to evaluate specialization and controller learning under varied semantics.

Figure 5 presents the effectiveness heatmap used to define ground-truth model-task interactions. Brighter colors indicate higher success probabilities. Claude-3.5 demonstrates high performance on *Analysis* (1.00) and *Text* (0.97) tasks but performs poorly on *Code* (0.02). GPT-3.5 shows strong results for *Explanation* (0.97), *Research* (0.94), and *Optimization* (0.89), whereas LLaMA-3 excels in *Planning* (0.95) and *Research* (0.97), while underperforming on *Text* (0.05). Mistral-7B demonstrates relatively balanced but moderate performance across task types, and GPT-4 remains consistently strong on high-structure tasks like *Math* and *Code*.

Figure 6 shows the prediction error dynamics across epochs. The blue line corresponds to the system with SPSA-based consensus controller adaptation, while the orange line represents a Round Robin baseline that does not adjust controller parameters.

Table 2 quantifies the Mean Squared Error (MSE) for various prediction tasks, highlighting the learning effectiveness of the SPSA-based consensus controllers. The table evaluates system performance by comparing predicted versus actual values across four key metrics: CPU Load reflects the disparity between forecasted and measured processor utilization; Memory Usage indicates the difference between anticipated and actual system memory consumption; Network Load quantifies the variance between predicted and real network bandwidth usage; and Queue Length captures the discrepancy between the forecasted and empirically observed number of tasks awaiting processing.

The SPSA-based consensus system demonstrates high prediction accuracy, particularly for execution time estimation. On average, it achieves a 46.08% reduction in mean squared error (MSE) across all metrics, confirming the adaptive learning capability of the controllers. Beyond accuracy, SPSA also improves resource efficiency: CPU usage decreases by 14.96%, memory consumption by 11.96%, network overhead by 20.32%, and queue lengths by 20.95% compared to the Round Robin baseline. These results highlight both the predictive precision and the operational efficiency of the proposed method.

SPSA-based consensus enables consistent error reduction from approximately 0.75 to below 0.1 within 50 iterations, demonstrating that the controller is able to learn accurate performance estimates over time. In contrast, the Round Robin system remains at a higher and fluctuating error level near 0.35, due to its inability to adapt to agent-task fit.

Figure 7 reports the average prediction error by task type. In most categories, the SPSA-based consensus controller yields lower or comparable error compared to the non-adaptive baseline. However, for certain types such as *Code*, SPSA shows a slightly higher average error. This is attributed to its early exploration phase, where controllers actively adjust predictions and may temporarily deviate before converging. These fluctuations are part of the learning process and tend to diminish in later epochs.

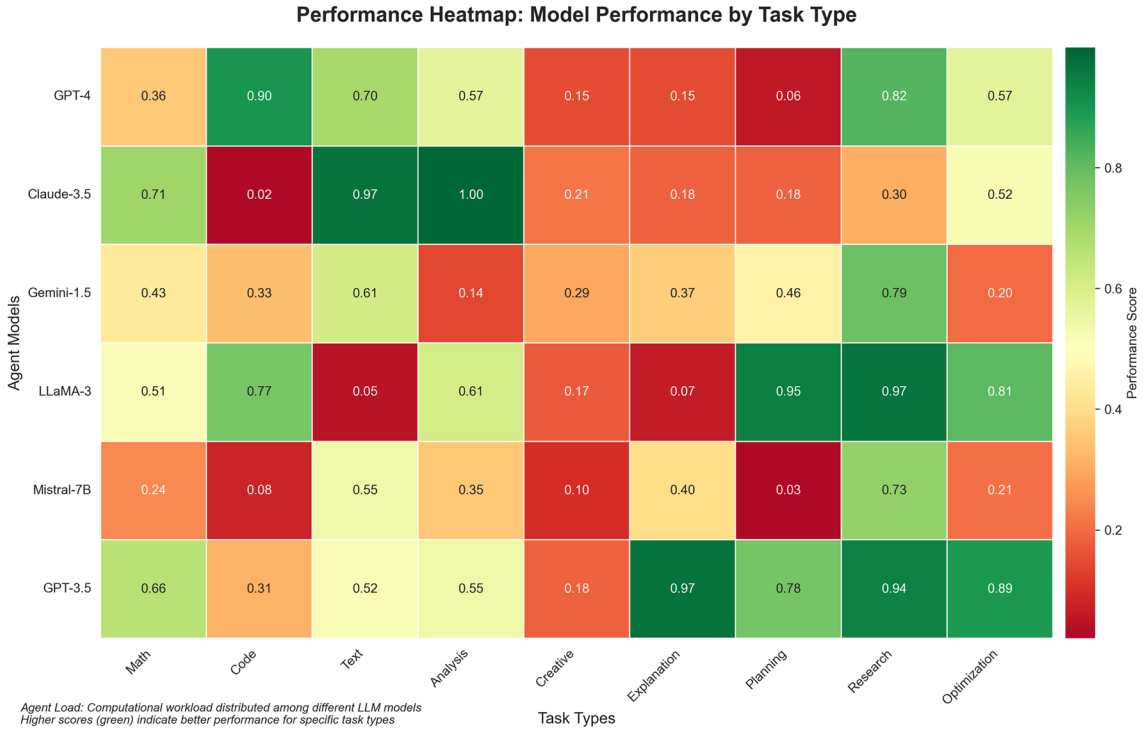



**Fig. 5**. Predefined agent effectiveness across task categories based on external studies.

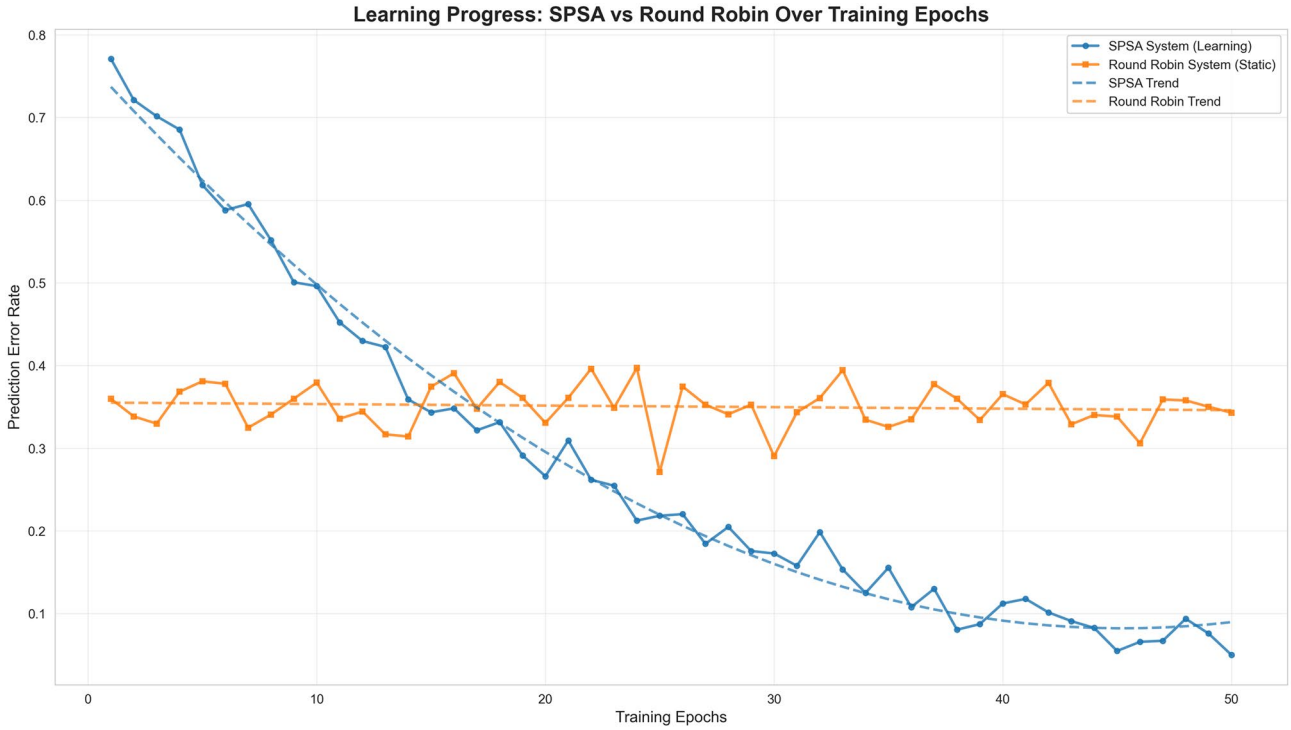



**Fig. 6**. Prediction error dynamics: SPSA vs. static (Round Robin) assignment.

| Prediction Target | SPSA MSE | Round Robin MSE | Improvement | Quality |
|---|---|---|---|---|
| CPU Load | 0.0347 | 0.0821 | 57.74% | High |
| Memory Usage | 0.0523 | 0.0891 | 41.32% | Good |
| Network Load | 0.0234 | 0.0456 | 48.68% | Good |
| Queue Length | 6.78 | 12.45 | 45.54% | Good |
| **Average MSE** | **1.374** | **2.548** | **46.08%** | **Significant** |

**Table 2**. Prediction accuracy: MSE comparison across different metrics.

The performance differences are statistically significant with 95% confidence intervals. A task-type analysis reveals $p < 0.05$ for Math, Analysis, Research, Planning, Debugging, and Summarization categories, confirming the specialized adaptation capability of the proposed system. Quantitatively, SPSA achieves a 68-fold reduction in execution time deviation, a 46.08% decrease in mean squared error (MSE), a 20.95% reduction in queue length, and 14.96% lower CPU usage compared to the Round Robin baseline. These improvements highlight both the stability and predictive accuracy of the SPSA-based consensus approach.

At the same time, the Round Robin strategy remains attractive in terms of simplicity and cost, offering up to 3.6× lower operational expense for creative and planning tasks. Thus, while SPSA is optimal for stability and accuracy, Round Robin may still be suitable when efficiency and minimal overhead take priority. In particular, under high-load conditions (task arrival rate above 1000 tasks/hour with response-time requirements below 0.01 seconds), SPSA is the only viable solution due to its robustness and adaptation speed, whereas in budget-constrained settings (cost below 1.0 unit per task) Round Robin remains the preferable choice despite lower stability.

The simulation code is available at the GitHub repository[38].

## Conclusion

This paper has introduced a decentralized architecture for task allocation in dynamic multi-agent systems, combining adaptive controllers, predictive modelling, and a local voting protocol (LVP). The system operates in an online setting with limited visibility and heterogeneous agents, and is designed to perform robustly in high-uncertainty environments.

The proposed framework integrates predictive models for task parameters, relevance- and availability-based agent selection, and decentralized coordination among controllers. We further enhance adaptability through simultaneous perturbation stochastic approximation (SPSA), enabling each controller to refine its model based on noisy feedback and local information. A key theoretical contribution is the proof of convergence under

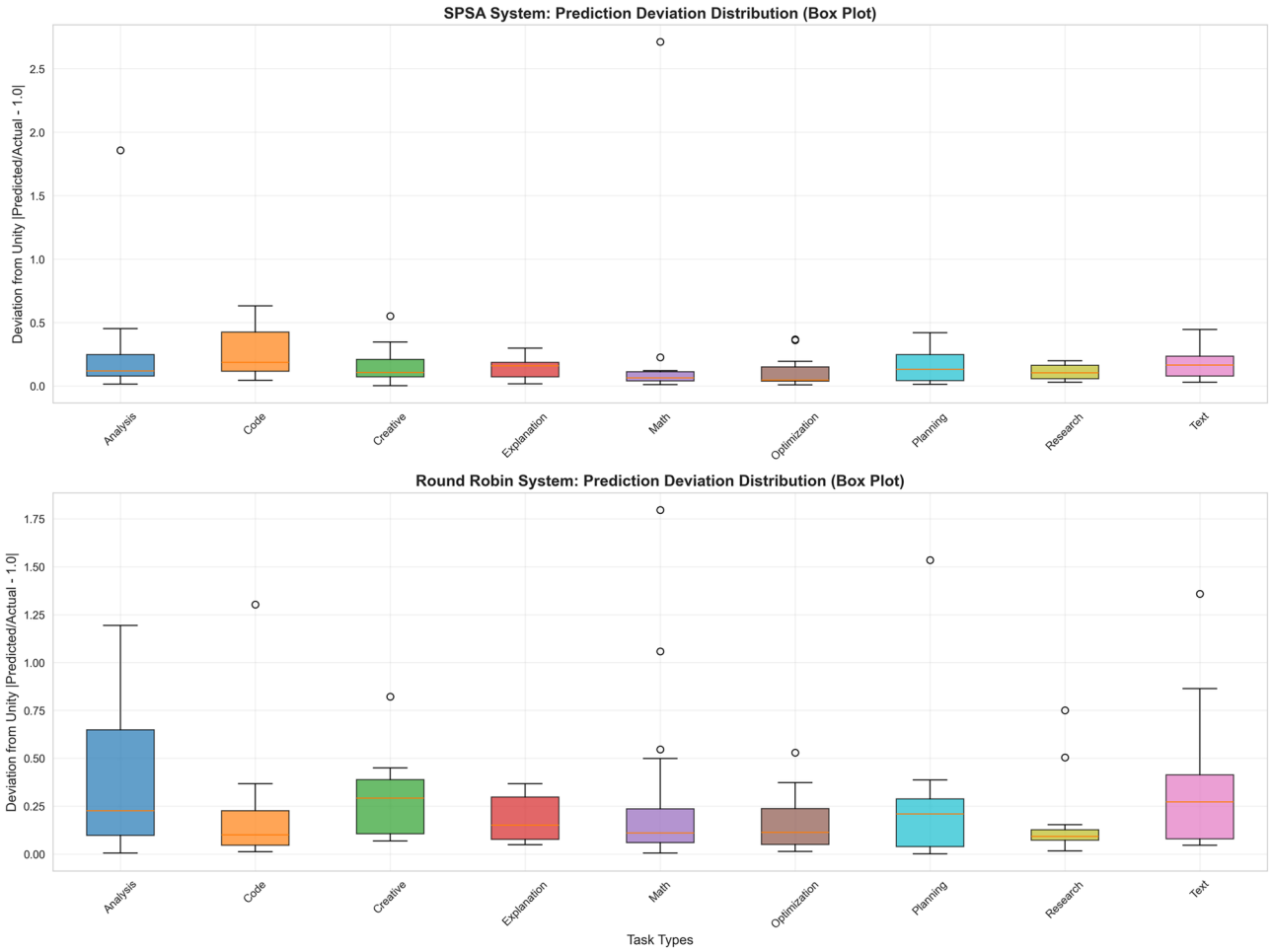



**Fig. 7**. Prediction error per task type: SPSA vs. Round Robin.

unknown-but-bounded noise and minimal assumptions, extending the applicability of SPSA-based consensus algorithms to two-layer decentralized settings.

Simulation results demonstrate the effectiveness of the architecture on real-world prompt-like tasks distributed among diverse large language models (LLMs). The system outperforms baseline approaches in both average performance and robustness across a variety of task categories. The second set of experiments confirms that the proposed adaptation mechanism enables stable online learning in the presence of structural drift and noise.

Beyond its empirical performance, the novelty of this work lies in explicitly addressing three persistent gaps in decentralized task allocation: (i) enabling online adaptation under behavioral drift and noisy feedback without centralized retraining, (ii) introducing a two-layer architecture with selective task visibility to support scalability and heterogeneity, and (iii) providing decentralized synchronization via SPSA-based consensus under bounded disturbances. Furthermore, by validating the framework on LLM-driven workloads, we demonstrate its applicability to emerging domains of decentralized knowledge work, where adaptability and robustness are critical. These aspects distinguish our approach from conventional auction-based, heuristic, or DRL-heavy methods and establish a foundation for scalable and resilient task allocation in dynamic multi-agent environments.

Future work will focus on integrating hierarchical control and reward-based feedback, enabling agent selection not only based on availability and predicted accuracy but also on long-term outcomes. Additionally, we plan to explore task allocation in resource-constrained or partially observable settings, further pushing the boundary of adaptive multi-agent coordination. While the current architecture focuses on decentralized adaptation and agent selection, it can be further extended toward orchestration-level control. By introducing additional coordination layers or hierarchical planning mechanisms, the system could support more complex task dependencies, agent negotiations, and global workflow management – enabling orchestration without compromising its decentralized foundation.

## Data availability

The data supporting the findings of this study are contained in the article.

Received: 8 August 2025; Accepted: 23 September 2025
Published online: 10 November 2025

## References


1. Braquet, M. & Bakolas, E. Greedy decentralized auction-based task allocation for multi-agent systems. *IFAC-PapersOnLine* **54**, 675–680. https://doi.org/10.1016/j.ifacol.2021.11.249 (2021).
2. Andreev, M. et al. Adaptive planning for supply chain networks. In *Holonic and Multi-Agent Systems for Manufacturing HoloMAS 2007, vol. 4659 of Lecture Notes in Computer Science* (eds Mařík, V. et al.) 215–224 (Springer, Berlin, Heidelberg, 2007). https://doi.org/10.1007/978-3-540-74481-8_21.
3. Bapat, A. et al. Distributed task allocation algorithms for multi-agent systems with very low communication. *IEEE Access* https://doi.org/10.1109/ACCESS.2022.3224146 (2022).
4. Ren, Y., Wang, Z. & Kumar, V. Hippo-mat: Hierarchical decentralized matching for scalable task allocation in multi-robot systems. *IEEE Trans. Robot.* **39**, 1372–1389. https://doi.org/10.1109/TRO.2022.3205617 (2023).
5. Jain, K., Singh, J. & Vishwanathan, S. V. N. Submodular meets streaming: Distributed algorithms for covering and matching on massive graphs. *IEEE Trans. Netw. Sci. Eng.* **6**, 116–127. https://doi.org/10.1109/TNSE.2018.2847885 (2019).
6. Skobelev, P., Zhilyaev, A., Laryukhin, V. B., Grachev, S. & Simonova, E. V. Ontology-based open multi-agent systems for adaptive resource management. In *Proceedings of the 12th International Conference on Agents and Artificial Intelligence (ICAART 2020)*, 127–135 (SciTePress, 2020).
7. Li, M., Zhang, K., Liu, Y., Song, F. & Li, T. Prescribed-time consensus of nonlinear multi-agent systems by dynamic event-triggered and self-triggered protocol. *IEEE Trans. Autom. Sci. Eng.* **22**, 16768–16779. https://doi.org/10.1109/TASE.2025.3578418 (2025).
8. Li, M., Long, Y., Zhang, K. & Li, T. Time-varying dynamic event-triggered-based prescribed-time leader-following consensus for multi-agent systems with input saturation. *Appl. Math. Comput.* **508**, 129630. https://doi.org/10.1016/j.amc.2025.129630 (2025) (**early access**).
9. Zhang, X.-M., Han, Q.-L., Ge, X. & Zhang, B.-L. Accumulative-error-based event-triggered control for discrete-time linear systems: A discrete-time looped functional method. *IEEE/CAA J. Autom. Sin.* https://doi.org/10.1109/JAS.2024.124476 (2024).
10. Buyya, R. & Srirama, S. N. *Fog and Edge Computing: Principles and Paradigms* (Wiley Telecom, 2019).
11. Khan, Z. A., Aziz, I. A., Osman, N. A. B. & Ullah, I. A review on task scheduling techniques in cloud and fog computing: Taxonomy, tools, open issues, challenges, and future directions. *IEEE Access* **11**, 143417–143445. https://doi.org/10.1109/ACCESS.2023.3343877 (2023).
12. Yang, Y., Ren, F. & Zhang, M. A decentralized multiagent-based task scheduling framework for handling uncertain events in fog computing. arXiv:2401.02219 (2024).
13. Yang, Z., Fu, J., Sun, Y. & Li, Y. A pseudo-trajectory homotopy method for umvs information collection iot system with an underwater communication constraint. *IEEE Internet Things J.* https://doi.org/10.1109/JIOT.2025.3596338 (2025).
14. Wang, J. & Li, D. Task scheduling based on a hybrid heuristic algorithm for smart production line with fog computing. *Sensors* **19**, 1023. https://doi.org/10.3390/s19051023 (2019).
15. Khan, F. U., Shah, I. A., Jan, S., Ahmad, S. & Whangbo, T. Machine learning-based resource management in fog computing: A systematic literature review. *Sensors* **25**, 687. https://doi.org/10.3390/s25030687 (2025).
16. Hosseinalipour, S., Brinton, C. G., Aggarwal, V., Dai, H. & Chiang, M. From federated to fog learning: Distributed machine learning over heterogeneous wireless networks. arXiv:2006.03594 (2020).
17. Netzer, A., Meisels, A. & Zivan, R. Distributed envy minimization for resource allocation. *Auton. Agents Multi-Agent Syst.* **30**, 364–402. https://doi.org/10.1007/s10458-015-9291-7 (2016).
18. Bhakhar, R. & Chhillar, R. S. Dynamic multi-criteria scheduling algorithm for smart home tasks in fog-cloud iot systems. *Sci. Rep.* **14**, 29957. https://doi.org/10.1038/s41598-024-81055-0 (2024).
19. Smith, R. G. The contract net protocol: High-level communication and control in a distributed problem solver. *IEEE Trans. Comput.* **C–29**, 1104–1113. https://doi.org/10.1109/TC.1980.1675516 (1980).
20. Liekna, A., Lavendelis, E. & Grabovskis, A. Experimental analysis of contract net protocol in multi-robot task allocation. *Appl. Comput. Syst.* **13**, 6–14. https://doi.org/10.2478/v10312-012-0001-7 (2012).
21. Ali, E. M., Lemma, F., Srinivasagan, R. & Abawajy, J. Efficient delay-sensitive task offloading to fog computing with multi-agent twin delayed deep deterministic policy gradient. *Electronics* **14**, 2169. https://doi.org/10.3390/electronics14112169 (2025).
22. Ratnabala, L., Peter, R., Fedoseev, A. & Tsetserukou, D. Hippo-mat: Decentralized task allocation using graphsage and multi-agent deep reinforcement learning. arXiv:2503.07662 (2025).
23. Xing, Y., Chen, C., Wu, J. & Chen, J. Reinforcement learning for efficient multi-agent task allocation in potential game model. *IEEE Trans. Artif. Intell.* https://doi.org/10.1109/TAI.2025.3562160 (2025).
24. Choppara, P. & Mangalampalli, S. S. A hybrid task scheduling technique in fog computing using fuzzy logic and deep reinforcement learning. *IEEE Access* **12**, 176363–176388. https://doi.org/10.1109/ACCESS.2024.3505546 (2024).
25. Jin, W. & Rezaeipanah, A. Dynamic task allocation in fog computing using enhanced fuzzy logic approaches. *Sci. Rep.* **15**, 18513. https://doi.org/10.1038/s41598-025-03621-4 (2025).
26. Ghafari, R. & Mansouri, N. An efficient task scheduling in fog computing using improved artificial hummingbird algorithm. *J. Comput. Sci.* **74**, 102152. https://doi.org/10.1016/j.jocs.2023.102152 (2023).
27. Maheswaran, R. T., Tambe, M., Bowring, E., Pearce, J. P. & Varakantham, P. Taking dcop to the real world: Efficient complete solutions for distributed multi-event scheduling. In *Proceedings of the Third International Joint Conference on Autonomous Agents and Multiagent Systems (AAMAS 2004)*, vol. 2, 310–317, https://doi.org/10.1109/AAMAS.2004.10067 (IEEE Computer Society, 2004).
28. Yokoo, M., Suzuki, K. & Hirayama, K. Secure distributed constraint satisfaction: Reaching agreement without revealing private information. In *Principles and Practice of Constraint Programming (CP 2002)*, vol. 2470 of *LNCS*, 387–401, https://doi.org/10.1007/3-540-46135-3_26 (Springer, 2002).
29. Khan, A. et al. Ecotasksched: a hybrid machine learning approach for energy-efficient task scheduling in iot-based fog-cloud environments. *Sci. Rep.* **15**, 12296. https://doi.org/10.1038/s41598-025-96974-9 (2025).
30. Amayuelas, A. et al. Self-resource allocation in multi-agent llm systems, https://doi.org/10.48550/arXiv.2504.02051. arXiv:2504.02051 (2025).
31. Granichin, O., Erofeeva, V., Ivanskiy, Y. & Jiang, Y. Simultaneous perturbation stochastic approximation-based consensus for tracking under unknown-but-bounded disturbances. *IEEE Trans. Autom. Control* **66**, 3710–3717 (2021).
32. Granichin, O. Linear regression and filtering under nonstandard assumptions (arbitrary noise). *IEEE Trans. Autom. Control* **49**, 1830–1837. https://doi.org/10.1109/TAC.2004.835585 (2004).
33. Erofeeva, V. et al. Multilevel modeling and control of dynamic systems. *Sci. Rep.* **14**, 27903. https://doi.org/10.1038/s41598-024-79279-1 (2024).
34. Mathias, D., Wu, A. & Dang, D. Analysis of evolved response thresholds for decentralized dynamic task allocation. *ACM Trans. Evol. Learn. Optim.* https://doi.org/10.1145/3530821 (2022).
35. Nayak, S. et al. Experimental comparison of decentralized task allocation algorithms under imperfect communication. *IEEE Robot. Autom. Lett.* **5**, 572–579. https://doi.org/10.1109/LRA.2019.2963646 (2020).

36. Amelina, N., Fradkov, A., Jiang, Y. & Vergados, D. J. Approximate consensus in stochastic networks with application to load balancing. *IEEE Trans. Inf. Theory* **61**, 1739–1752 (2015).
37. Erofeeva, V., Granichin, O. & Volodina, E. Accelerated decentralized load balancing in multi-agent networks. *IEEE Access* **12**, 161954–161967. https://doi.org/10.1109/ACCESS.2024.3488399 (2024).
38. Chernikov, K. & Tarasova, E. Code for "decentralized adaptive task allocation for dynamic multi-agent systems". (accessed 8 Aug 2025). https://github.com/newpotatato/Multi_agent_system (2025).

## Author contributions
Conceptualization, E.T.; methodology, O.G., E.T. and V.E.; software, K.C. and E.T.; writing—original draft preparation, E.T.; writing—review and editing, V.E, and O.G.; visualization, K.C. All authors reviewed the manuscript.

## Funding
This work was supported by the Ministry of Economic Development of the Russian Federation (Agreement No. 139-15-2025-007, dated April 16, 2025; ID: 000000C313925P3O0002).

## Declarations

## Competing interests
The authors declare no competing interests.

## Additional information
**Supplementary Information** The online version contains supplementary material available at https://doi.org/10.1038/s41598-025-21709-9.

**Correspondence** and requests for materials should be addressed to E.T.

**Reprints and permissions information** is available at www.nature.com/reprints.

**Publisher's note** Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

**Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

© The Author(s) 2025